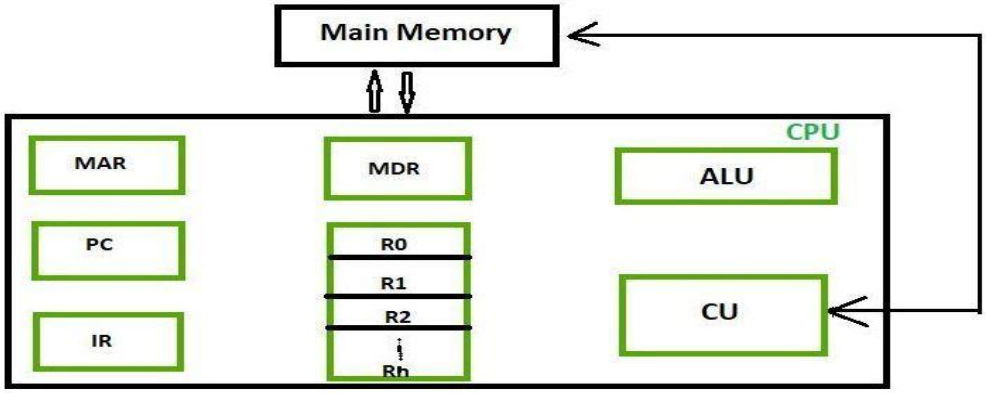
CSPC402 COMPUTER ORGANIZATION AND ARCHITECTURE

UNIT II (INSTRUCTION SET ARCHITECTURE (RISC-V): CPU registers, Instruction format and encoding, addressing modes, Instruction set, Instruction types, Instruction Decoding and Execution, Basic Instruction cycle, Reduced Instruction Set Computer (RISC), Complex Instruction Set Computer (CISC), RISC-V instructions - X86 Instruction set.

CPU registers

In Computer Architecture, the Registers are very fast computer memory that is used to execute programs and operations efficiently. This is done by giving access to commonly used values, i.e., the values that are at the point of operation/execution at that time. So, for this purpose, there are several different classes of CPU registers that work in coordination with the computer memory to run operations efficiently.

The sole purpose of having a register is fast retrieval of data for processing by CPU. Though accessing instructions from RAM is comparatively faster with a hard drive, it still isn't enough for the CPU. For even better processing, there are memories in the CPU that can get data from RAM which are about to be executed beforehand. After registers, we have cache memory, which is faster but less fast than registers.



CPU registers are essential for the execution of instructions, and understanding the different types, such as general-purpose, segment, and special-purpose registers, is crucial for studying computer architecture.

Different Types of CPU Registers

There are several types of registers available in the CPU for different purposes let's discuss each one by one:

- **Accumulator** : This is the most frequently used register used to store data taken from memory. It is indifferent numbers in different microprocessors.

- **Memory Address Registers (MAR)** : It holds the address of the location to be accessed from memory. MAR and MDR (Memory Data Register) together facilitate the communication of the CPU and the main memory.

- **Memory Data Registers (MDR)** : It contains data to be written into or to be read out from the addressed location.

- **General Purpose Registers** : These are numbered as R0, R1, R2….Rn-1, and used to store temporary data during any ongoing operation. Its content can be accessed by assembly programming. Modern CPU architectures tend to use more GPR so that register-to-register addressing can be used more, which is comparatively faster than other addressing modes .

- **Program Counter (PC)** : Program Counter (PC) is used to keep the track of the execution of the program. It contains the memory address of the next instruction to be fetched. PC points to the address of the next instruction to be fetched from the main memory when the previous instruction has been successfully completed. Program Counter (PC) also functions to count the number of instructions. The incrementation of PCs depends on the type of architecture being used. If we are using a 32-bit architecture, the PC gets incremented by 4 every time to fetch the next instruction.

- **Instruction Register (IR):** The IR holds the instruction which is just about to be executed. The instruction from the PC is fetched and stored in IR. As soon as the instruction is placed in IR, the CPU starts executing the instruction, and the PC points to the next instruction to be executed.

- **Stack Pointer (SP): The stack** PCs pointer points to the top of the stack, which is a part of the memory used to store function calls and other operations.

- **Flag Register:** A flag register , also known as a status register or condition code register, is a special type of register in a computer's central processing unit (CPU) used to indicate the status of the CPU or the outcome of various operations such as Zero Flag, Carry flag, Sign Flag, Overflow Flag, Parity Flag, Auxiliary Carry Flag, and Interrupt Enable Flag.

- **Condition code register (CCR):** Condition code registers contain different flags that indicate the status of any operation. For instance, let's suppose an operation caused the creation of a negative result or zero, then these flags are set high accordingly. and the flags are

1. Carry C: Set to 1 if an add operation produces a carry or a subtract operation produces a borrow; otherwise cleared to 0.

2. Overflow V: Useful only during operations on signed integers.

3. Zero Z: Set to 1 if the result is 0, otherwise cleared to 0.
4. Negate N: Meaningful only in signed number operations. Set to 1 if a negative result is produced.
5. Extend X: Functions as a carry for multiple precision arithmetic operations.

These are generally decided by ALU.

So, these are the different registers that are operating for a specific purpose.

Size of CPU Register

The number and size of the register in the CPU depend on the processor design and can have an important impact on its performance and capabilities on the CPU. Now, let's discuss the different sizes of the register available in the CPU:

- **8-bit registers:** These can store 8 bits of data, which is the same as 1 byte. They are used for simple arithmetic and data manipulation.
- **16-bit registers:** These hold 16 bits of data, or 2 bytes. These registers are found in older processors or certain systems that need 16-bit operations.
- **32-bit registers:** These can store 32 bits of data or 4 bytes. They are common in many processors and can handle larger data sizes and more complex calculations than 8-bit or 16-bit registers.
- **64-bit registers:** These hold 64 bits of data, or 8 bytes. Modern processors often use these, providing more computational power and better memory-addressing capabilities.

Most modern computers use 32-bit or 64-bit registers, and that's why we call them 32-bit or 64-bit processors. This terminology refers to the width of the processor's registers and how much data it can handle at once.

In some specialized processors, you might see even larger registers like 128-bit or 256-bit. These are used for specific tasks such as vector processing or cryptography where dealing with large data sets and parallel processing is important.

Purpose of Registers

Registers play a very important role in computers, and they are used by CPU for various purposes.

- **Storing Instruction:** Registers are used to store the instruction from programs before the CPU follows them. This helps the computer quickly find and follow the steps it needs to take.

- **Holding Answer:** When the computer does math calculations or other tasks, the register stores the temporary answer.
- **Quick Access to Important Stuff:** Registers are like the computer's quick-access shelves. They keep important things nearby, so the computer can grab them fast without going far away to get them. It's like keeping your favorite tools on a shelf right next to you, instead of in a faraway closet.

INSTRUCTION FORMAT AND ENCODING

Instruction formats refer to the way instructions are encoded and represented in machine language. There are several types of instruction formats, including zero, one, two, and three-address instructions.
Each type of instruction format has its own advantages and disadvantages in terms of code size, execution time, and flexibility. Modern computer architectures typically use a combination of these formats to provide a balance between simplicity and power.

Different Types of Instruction Fields

A computer performs a task based on the instructions provided. Instructions in computers are comprised of groups called fields. These fields contain different information for computers which are all written in 0s and 1s. Each field has a different significance or meaning, based on which a CPU decides what to perform. The most common fields are:

- The operation field specifies the operation to be performed, like addition.
- Address field which contains the location of the operand, i.e., register or memory location.
- Mode field which specifies how operand is to be founded.

An instruction is of variable length depending upon the number of addresses it contains. Generally, CPU organization is of three types based on the number of address fields:

- Single Accumulator organization
- General register organization
- Stack organization

In the first organization, the operation is performed using a special register called the accumulator. In the second, multiple registers are used for the computation purpose. In the third organization, the work on stack basis operation due to which it does not contain any address field. We generally see a blend of various organizations applied

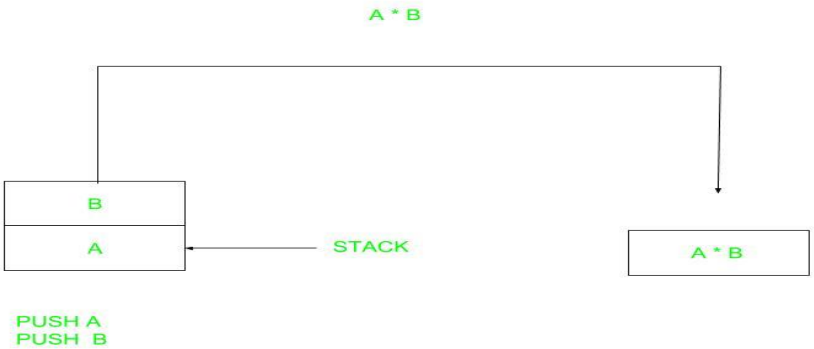
Types of Instructions

Based on the number of addresses, instructions are classified as:

NOTE: We will use the X = (A+B)*(C+D) expression to showcase the procedure.

Zero Address Instructions

These instructions do not specify any operands or addresses. Instead, they operate on data stored in registers or memory locations implicitly defined by the instruction. For example, a zero-address instruction might simply add the contents of two registers together without specifying the register names.



A stack-based computer does not use the address field in the instruction. To evaluate an expression, it is first converted to reverse Polish Notation i.e. Postfix Notation.

Expression:X=(A+B)*(C+D)
Postfixed:X=AB+CD+*
TOP means top of stack
M[X] is any memory location

PUSH	A	TOP = A
PUSH	B	TOP = B
ADD		TOP = A+B
PUSH	C	TOP = C
PUSH	D	TOP = D

PUSH	A	TOP = A
ADD		TOP = C+D
MUL		TOP = (C+D)*(A+B)
POP	X	M[X] = TOP

One Address Instructions

These instructions specify one operand or address, which typically refers to a memory location or register. The instruction operates on the contents of that operand, and the result may be stored in the same or a different location. For example, a one-address instruction might load the contents of a memory location into a register.

This uses an implied ACCUMULATOR register for data manipulation. One operand is in the accumulator and the other is in the register or memory location. Implied means that the CPU already knows that one operand is in the accumulator so there is no need to specify it.

opcode	operand/address of operand	mode

One Address Instruction

Expression:X=(A+B)*(C+D)
AC is accumulator
M[] is any memory location
M[T] is temporary location

LOAD	A	AC = M[A]
ADD	B	AC = AC + M[B]

LOAD	A	AC = M[A]
STORE	T	M[T] = AC
LOAD	C	AC = M[C]
ADD	D	AC = AC + M[D]
MUL	T	AC = AC * M[T]
STORE	X	M[X] = AC

Two Address Instructions

These instructions specify two operands or addresses, which may be memory locations or registers. The instruction operates on the contents of both operands, and the result may be stored in the same or a different location. For example, a two-address instruction might add the contents of two registers together and store the result in one of the registers.

This is common in commercial computers. Here two addresses can be specified in the instruction. Unlike earlier in one address instruction, the result was stored in the accumulator, here the result can be stored at different locations rather than just accumulators, but require more number of bit to represent the address.

opcode	Destination address	Source address	mode

Two Address Instruction

Here destination address can also contain an operand.

Expression: X = (A+B)*(C+D)
R1, R2 are registers
M[] is any memory location

MOV	R1, A	R1 = M[A]
ADD	R1, B	R1 = R1 + M[B]
MOV	R2, C	R2 = M[C]
ADD	R2, D	R2 = R2 + M[D]
MUL	R1, R2	R1 = R1 * R2
MOV	X, R1	M[X] = R1

Three Address Instructions

These instructions specify three operands or addresses, which may be memory locations or registers. The instruction operates on the contents of all three operands, and the result may be stored in the same or a different location. For example, a three-address instruction might multiply the contents of two registers together and add the contents of a third register, storing the result in a fourth register.

This has three address fields to specify a register or a memory location. Programs created are much short in size but number of bits per instruction increases. These instructions make the creation of the program much easier but it does not mean that program will run much faster because now instructions only contain more information but each micro-operation (changing the content of the register, loading address in the address bus etc.) will be performed in one cycle only.

opcode	Destination address	Source address	Source address	mode

Three Address Instruction

Expression: X = (A+B)*(C+D)
R1, R2 are registers
M[] is any memory location

ADD	R1, A, B	R1 = M[A] + M[B]
ADD	R2, C, D	R2 = M[C] + M[D]
MUL	X, R1, R2	M[X] = R1 * R2

INSTRUCTION SET

An instruction is a set of codes that the computer processor can understand. The code is usually in 1s and 0s, or machine language. It contains instructions or tasks that control the movement of bits and bytes within the processor.

Example of some instruction sets

ADD − Add two numbers together.

JUMP − Jump to designated RAM address.

LOAD − Load information from RAM to the CPU.

Types of Instruction Set

Generally, there are two types of instruction set used in computers.

Reduced Instruction set Computer (RISC)

A number of computer designers recommended that computers use fewer instructions with simple constructs so that they can be executed much faster within the CPU without having to use memory as often. This type of computer is called a Reduced Instruction Set Computer.

The concept of RISC involves an attempt to reduce execution time by simplifying the instruction set of computers.

Complex Instruction Set Computer (CISC)

CISC is a computer where a single instruction can perform numerous low-level operations like a load from memory and a store from memory, etc. The CISC attempts to minimize the number of instructions per program but at the cost of an increase in the number of cycles per instruction.

The design of an instruction set for a computer must take into consideration not only machine language constructs but also the requirements imposed on the use of high level programming languages.

ADDRESSING MODES

The term addressing modes refers to the way in which the operand of an instruction is specified. The addressing mode specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually executed.

Thus, the different ways of specifying the location of an operand in an instruction are called as addressing modes:

1. Implied/ Implicit Addressing Mode
2. Stack addressing Mode
3. Immediate Addressing Mode
4. Direct Addressing Mode
5. Indirect Addressing Mode
6. Register Direct Addressing Mode
7. Register Indirect Addressing Mode
8. Relative Addressing Mode
9. Indexed Addressing Mode
10. Base Register Addressing Mode
11. Auto-increment Addressing Mode
12. Auto-decrement Addressing Mode

Addressing modes for 8086 instructions are divided into two categories:

1) Addressing modes for data

2) Addressing modes for branch

The 8086 memory addressing modes provide flexible access to memory, allowing you to easily access variables, arrays, records, pointers, and other complex data types. The key to good assembly language programming is the proper use of memory addressing modes.

An assembly language program instruction consists of two parts

Opcode	Operand

The memory address of an operand consists of two components:

IMPORTANT TERMS

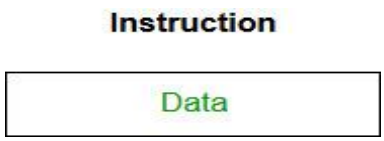
- **Starting address** of memory segment.
- **Effective address or Offset**: An offset is determined by adding any combination of three address elements: **displacement, base and index.**

- **Displacement:** It is an 8 bit or 16 bit immediate value given in the instruction.
- **Base**: Contents of base register, BX or BP.
- **Index**: Content of index register SI or DI.

According to different ways of specifying an operand by 8086 microprocessor, different addressing modes are used by 8086.

Addressing modes used by 8086 microprocessor are discussed below:

- **Implied mode:** In implied addressing the operand is specified in the instruction itself. In this mode the data is 8 bits or 16 bits long and data is the parts of instruction.Zero address instruction are designed with implied addressing mode.



CLC (used to reset Carry flag to 0)

- **Immediate addressing mode (symbol #):**In this mode data is present in address field of instruction .Designed like one address instruction format.
  Note: Limitation in the immediate mode is that the range of constants are restricted by size of address field.

Example: MOV AL, 35H (move the data 35H into AL register)

- **Register mode:** In register addressing the operand is placed in one of 8 bit or 16 bit general purpose registers. The data is in the register that is specified by the instruction. Here one register reference is required to access the data.

Example: MOV AX,CX (move the contents of CX register to AX register)

- **Register Indirect mode**: In this addressing the operand's offset is placed in any one of the registers BX,BP,SI,DI as specified in the instruction. The effective address of the data is in the base register or an index register that is specified by the instruction.
  Here two register reference is required to access the data.

  The 8086 CPUs let you access memory indirectly through a register using the register indirect addressing modes.

  MOV AX, [BX](move the contents of memory location s

  addressed by the register BX to the register AX)

Auto Indexed (increment mode): Effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next consecutive memory location.**(R1)+**.
Example:

Add R1, (R2)+ // OR

R1 = R1 +M[R2]

R2 = R2 + d

Auto indexed (decrement mode): Effective address of the operand is the contents of a register specified in the instruction. Before accessing the operand, the contents of this register are automatically decremented to point to the previous consecutive memory location. –**(R1)**

Example:

Add R1,-(R2) //OR

R2 = R2-*d*

R1 = R1 + M[R2]

Direct addressing/ Absolute addressing Mode (symbol [ ]): The operand's offset is given in the instruction as an 8 bit or 16 bit displacement element. In this addressing mode the 16 bit effective address of the data is the part of the instruction.

Example:ADD AL,[0301] //add the contents of offset address 0301 to AL

Indirect addressing Mode (symbol @ or ()):In this mode address field of instruction contains the address of effective address. Here two references are required.
1st reference to get effective address.
2nd reference to access the data.

Based on the availability of Effective address, Indirect mode is of two kind:

1. Register Indirect:In this mode effective address is in the register, and corresponding register name will be maintained in the address field of an instruction.
   Here one register reference,one memory reference is required to access the data.

2. Memory Indirect:In this mode effective address is in the memory, and corresponding memory address will be maintained in the address field of an instruction.

Indexed addressing mode: The operand's offset is the sum of the content of an index register SI or DI and an 8 bit or 16 bit displacement.

Example:MOV AX, [SI +05]

Based Indexed Addressing: The operand's offset is sum of the content of a base register BX or BP and an index register SI or DI.

Example: ADD AX, [BX+SI]

STEPS INVOLVED IN INSTRUCTION FETCH AND DECODE

Let's take a look at the steps involved in the instruction cycle from the beginning to the end:

Step 1: Fetch Phase – The first phase of the instruction cycle

Here, the sequence counter (SC) is set to zero at the start of the instruction cycle.

SC – 0

Step 2: Fetch phase occurring at Clock Pulse (T-0)

Here, the instruction cycle stores the address of the next instruction in the program counter (PC) register. The PC's content (address) is assigned to the address register in the first clock cycle (T-0) (AR).

PC – AR

Step 3: Fetch phase occurring at Clock Pulse 1

In the next clock cycle (T-1), the instruction is read from memory and loaded into the instruction register (IR), while the program counter (PC) is also incremented by one. The program counter (PC) now points to the memory address of the next instruction to be retrieved.

PC – PC+1 AND IR -M [AR]

The Instruction Register (IR) is a 16-bit register capable of storing instructions in 16-bit format. The three components of the 16-bit instruction format (0 to 11 bit) are:

1. Addressing Mode (15th bit)
2. OPCODE (12, 13, 14 bit)
3. Operand Address

Step 4: Decode Phase occurring at Clock Pulse (T-2)

The CPU's control unit decodes the program instruction once it is loaded into the instruction register (IR). A Decode Phase is the second phase of the instruction cycle.

The instruction is decoded by the CPU's control unit based on the operation code (OPCODE), determined by three bits (bit 12, 13, 14).

Step 5: Decode Instruction Type occurring at Clock Pulse (T-2)

The CPU's control unit must first decode the type of instruction. The three types of education are:

1. Memory reference instruction
2. Register reference instruction
3. Input/output instruction.

The 3 X 8 decoder determines the type of instruction. The D7 value determines the type of instruction. The three-bit OPCODE can have eight distinct values, starting with (DO, D1, D2, ….., D6, D7). Alternatively, the OPCODE can have values ranging from DO – 000 to D7 111.

Step 6: Decode Addressing Mode occurring at Clock Pulse (T-3)

If D7 is set to zero, the instruction type is a memory reference. When D7 is set to 1, the instruction type can be a register reference type or an input/output instruction. The instruction type is register reference type if D7 equals one and I equals 0. If D7 and I are both 1, the instruction type is input/output.

After deciding on the type of instructions and making a subsequent decision, the decode phase concludes. The value of I in the 15th bit determines the decode phase.

BASIC INSTRUCTION CYCLE

an instruction cycle, also known as a fetch-decode-execute cycle, is the basic operation performed by a central processing unit (CPU) to execute an instruction. The instruction cycle consists of several steps, each of which performs a specific function in the execution of the instruction. The major steps in the instruction cycle are:

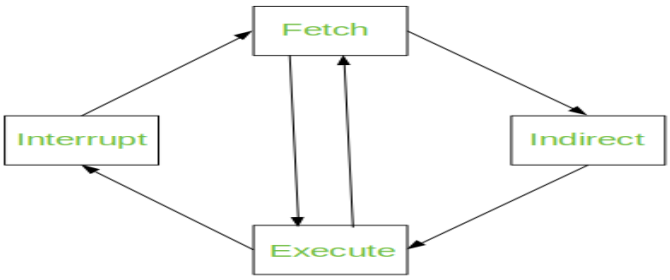
Fetch: In the fetch cycle, the CPU retrieves the instruction from memory. The instruction is typically stored at the address specified by the program counter (PC). The PC is then incremented to point to the next instruction in memory.

Decode: In the decode cycle, the CPU interprets the instruction and determines what operation needs to be performed. This involves identifying the opcode and any operands that are needed to execute the instruction.

Execute: In the execute cycle, the CPU performs the operation specified by the instruction. This may involve reading or writing data from or to memory, performing arithmetic or logic operations on data, or manipulating the control flow of the program.

The Instruction Cycle:

Each phase of Instruction Cycle can be decomposed into a sequence of elementary micro-operations. In the above examples, there is one sequence each for the Fetch, Indirect, Execute and Interrupt Cycles.



The Instruction Cycle

The Fetch Cycle:

At the beginning of the fetch cycle, the address of the next instruction to be executed is in the Program Counter(PC).

MAR	
MBR	
PC	0000000001100100
IR	
AC	

BEGINNING

Step 1: The address in the program counter is moved to the memory address register(MAR), as this is the only register which is connected to address lines of the system bus.

MAR	0000000001100100
MBR	
PC	0000000001100100
IR	
AC	

FIRST STEP

Step 2: The address in MAR is placed on the address bus, now the control unit issues a READ command on the control bus, and the result appears on the data bus and is then copied into the memory buffer register(MBR). Program counter is incremented by one, to get ready for the next instruction. (These two action can be performed simultaneously to save time)

MAR	0000000001100100
MBR	0001000000100000
PC	0000000001100101
IR	
AC	

SECOND STEP

Step 3: The content of the MBR is moved to the instruction register(IR).

MAR	0000000001100100
MBR	0001000000100000
PC	0000000001100100
IR	0001000000100000
AC	

FIRST STEP

Thus, a simple Fetch Cycle consist of three steps and four micro-operation. Symbolically, we can write these sequence of events as follows:-

$$t1 : MAR \leftarrow PC$$
$$t2 : MBR \leftarrow MEMORY$$
$$PC \leftarrow (PC) + I$$
$$t3 : IR \leftarrow (MBR)$$

Here 'I' is the instruction length. The notations (t1, t2, t3) represents successive time units. We assume that a clock is available for timing purposes and it emits regularly spaced clock pulses. Each clock pulse defines a time unit. Thus, all time units are of equal duration. Each micro-operation can be performed within the time of a single time unit.

instruction is fetched, the next step is to fetch source operands. Source Operand is being fetched by indirect addressing(it can be fetched by any addressing mode, here its done by indirect addressing). Register-based operands need not be fetched. Once the opcode is executed, a similar process may be needed to store the result in main memory. Following micro-operations takes place:-

$$t1 : MAR \leftarrow (IR(ADDRESS))$$
$$t2 : MBR \leftarrow MEMORY$$
$$t3 : IR(ADDRESS) \leftarrow (MBR(ADDRESS))$$

Step 1: The address field of the instruction is transferred to the MAR. This is used to fetch the address of the operand.
Step 2: The address field of the IR is updated from the MBR.(So that it now contains a direct addressing rather than indirect addressing)
Step 3: The IR is now in the state, as if indirect addressing has not been occurred.

Execute Cycle

The data transfer for implementation takes place in two methods are as follows:

Processor-memory − The data sent from the processor to memory or from memory to processor.

Processor-Input/Output − The data can be transferred to or from a peripheral device by the transfer between a processor and an I/O device.

In the execute cycle, the processor implements the important operations on the information, and consistently the control calls for the modification in the sequence of data implementation. These two methods associate and complete the execute cycle.

consider an add instruction:

```
ADD  R , X
```

Here, this instruction adds the content of location X to register R. Corresponding micro-operation will be:-

```
t1 : MAR     ←(IR(ADDRESS))
t2 : MBR     ←MEMORY
t3 : R       ←(R) + (MBR)
```

We begin with the IR containing the ADD instruction.
Step 1: The address portion of IR is loaded into the MAR.
Step 2: The address field of the IR is updated from the MBR, so the reference memory location is read.
Step 3: Now, the contents of R and MBR are added by the ALU.

The Interrupt Cycle:
At the completion of the Execute Cycle, a test is made to determine whether any enabled

interrupt has occurred or not. If an enabled interrupt has occurred then Interrupt Cycle occurs. The nature of this cycle varies greatly from one machine to another.

Lets take a sequence of micro-operation:-

```
t1 : MBR        <---  (PC)
t2 : MAR        <---  SAVE_ADDRESS
     PC         <---  ROUTINE_ADDRESS
t3 : MEMORY     <---  (MBR)
```

Step 1: Contents of the PC is transferred to the MBR, so that they can be saved for return.
Step 2: MAR is loaded with the address at which the contents of the PC are to be saved.
PC is loaded with the address of the start of the interrupt-processing routine.
Step 3: MBR, containing the old value of PC, is stored in memory.

REDUCED INSTRUCTION SET COMPUTER (RISC)

Reduced Instruction Set Computing is referred to as "RISC" in the context of computer architecture. It is a key idea that has profoundly influenced the structure and operation of contemporary processors. RISC design, which emphasizes simplicity, efficiency, and streamlined execution, marks a change from the prior complex instruction set computing (CISC) methodology.

Examples of RISC processors: x IBM RS6000, MC88100. x DEC's Alpha 21064, 21164 and 21264 processors

principles of RISC architecture

- Single-cycle execution: In most traditional central processing unit (CPU) designs, the peak possible execution rate is one instruction per basic machine cycle. For a given technology, the cycle time is determined by the slowest instruction in the instruction set. RISC designs aim to execute most instructions in a single cycle, increasing the processor's overall speed.

- Load-store architecture: RISC architectures use a load-store architecture, meaning only load and store instructions can access memory. All other instructions must operate on data in registers. This simplifies the instruction set and reduces the number of memory accesses required.

- Simple instructions: RISC architectures use simple instructions that can be executed quickly. This reduces the complexity of the processor and allows it to operate at a higher clock speed.

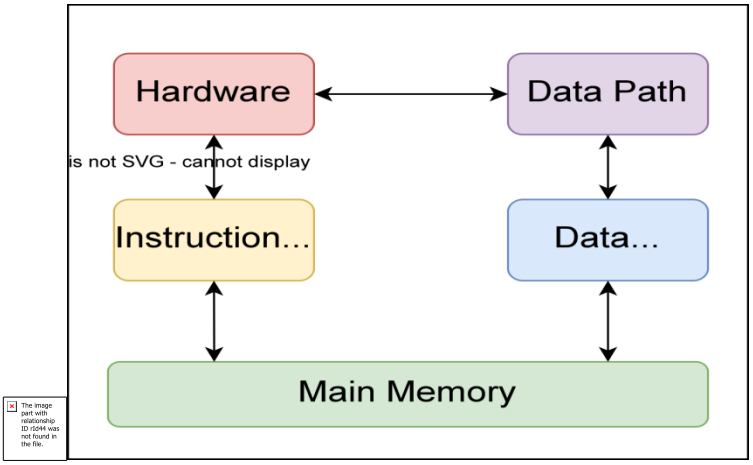
- Large register set: RISC architectures have many registers, which reduces the number of memory accesses required and allows for more efficient use of the processor's resources.
- Pipelining: RISC architectures use pipelining to increase the speed of instruction execution. Pipelining allows multiple instructions to be executed simultaneously, increasing the processor's overall throughput.

The RISC architecture diagram

The hardware of RISC architecture is designed to execute the instruction quickly, which is possible because of the more precise and smaller number of instructions and a large number of registers.

In RISC, the data path is used to store and manipulate data in a computer. It is responsible for managing data within the processor and its movement between the processor and the memory.

The processor uses a cache to reduce the access time to the main memory. The instruction cache is beneficial for retrieving and storing the data of frequently used instructions. It speeds up the process of instruction execution. The data cache provides storage for frequently used data from the main memory.



RISC instruction set

In RISC, the length of the instruction format is fixed. It took one-word memory. The fixed size of the instruction format benefits the program counter as it knows that the next instruction starts from where due to the fixed length of all instructions. In RISC, each instruction requires only one clock execution cycle. In addition, RISC architectures are designed to be highly scalable and accommodate a more significant number of instructions.

Two key characteristics of RISC instruction sets are:

• Each instruction fits in a single word.

• A load/store architecture is used, in which

– Memory operands are accessed only using Load and Store instructions.

– All operands involved in an arithmetic or logic operation must either be in processor registers, or one of the operands may be given explicitly within the instruction word. Load instructions are of the form

Load destination, source

The operation of adding two numbers is a fundamental capability in any computer. The statement **C = A + B** in a high-level language program instructs the computer to add the current values of the two variables called A and B, and to assign the sum to a third variable, C.

Hence, the above high-level language statement requires the action

C ← [A]+[B]

to take place in the computer. To carry out this action, the contents of memory locations A and B are fetched from the memory and transferred into the processor where their sum is computed. This result is then sent back to the memory and stored in location C. The required action can be accomplished by a sequence of simple machine instructions.

We choose to use registers R2, R3, and R4 to perform the task with four instructions:

Load R2, A

Load R3, B

Add R4, R2, R3

Store R4, C

Add is a three-operand, or a three-address, instruction of the form

Add destination, source1, source2

The Store instruction is of the form

Store source, destination

where the source is a processor register and the destination is a memory location.

Note that we can accomplish the desired addition by using only two registers, R2 and R3, if one of the source registers is also used as the destination for the result. In this case the addition would be performed as

Add R3, R2, R3

and the last instruction would become

Store R3, C

Disadvantages of RISC architecture

- Complex instructions and addressing modes: It isn't easy to process complex instructions and complex addressing modes in the RISC architecture.
- Direct memory-to-memory transfer: It uses load-store architecture. Hence, it doesn't allow a direct memory-to-memory transfer.
- Increase in the program length: RISC architecture has a small and simple instruction set. However, it requires more instruction to operate CISC architecture, increasing the program's length.

Advantages of RISC architecture

The advantages of RISC architecture are as follows:

- Simplified instruction set: RISC architecture uses a small instruction set that is highly optimized and simple; instruction executes quickly.
- Format length is fixed: In RISC architecture format length of instruction is fixed, which makes the execution or decoding of instructions faster.
- Register-based architecture: It stores data in the register within the processor, which is frequently used. This improves the performance because it reduces the number of memory access.
- Fewer cycles: In RISC architecture, the instruction requires less number of cycles to execute because of the simple instruction set.
- Load-Store architecture: RISC architecture uses load-store architecture, which means memory access differs from logical and arithmetic operations. Therefore, the processor's resources are used more efficiently, improving performance.

CISC (COMPLEX INSTRUCTION SET COMPUTER)

CISC stands for Complex Instruction Set Computer. If the control unit contains a number of microelectronic circuitry to generate a set of control signals and each micro-circuitry is activated by a microcode, this design approach is called CISC design.

Examples of CISC processors are: x Intel 386, 486, Pentium, Pentium Pro, Pentium II, Pentium III x Motorola's 68000, 68020, 68040, etc.

Features of CISC Processors:

The standard features of CISC processors are listed below:

- CISC chips have a large amount of different and complex instructions.
- CISC machines generally make use of complex addressing modes
- Different machine programs can be executed on CISC machine.
- CISC machines uses micro-program control unit.
- CISC processors are having limited number of registers.

CISC Instruction Sets

One key difference is that CISC instruction sets are not constrained to the load/store architecture, in which arithmetic and logic operations can be performed only on operands that are in processor registers. Another key difference is that instructions do not necessarily have to fit into a single word. Some instructions may occupy a single word, but others may span multiple words.

Instructions in modern CISC processors typically do not use a three-address format.

Most arithmetic and logic instructions use the two-address format

Operation destination, source

An Add instruction of this type is

Add B, A

which performs the operation B ← [A] + [B] on memory operands. When the sum is calculated, the result is sent to the memory and stored in location B, replacing the original contents of this location. This means that memory location B is both a source and a destination.

Consider again the task of adding two numbers

C = A + B

The task can be performed by using another two-address instruction that copies the contents of one memory location into another. Such an instruction is

Move C, B

which performs the operation C ← [B], leaving the contents of location B unchanged. The operation C ← [A] + [B] can now be performed by the two-instruction sequence

Move C, B

Add C, A

Observe that by using this sequence of instructions the contents of neither A nor B locations are overwritten.

In some CISC processors one operand may be in the memory but the other must be in a register. In this case, the instruction sequence for the required task would be

Move Ri, A

Add Ri, B

Move C, Ri

The general form of the Move instruction is

Move destination, source

where both the source and destination may be either a memory location or a processor register.

CPU Performance

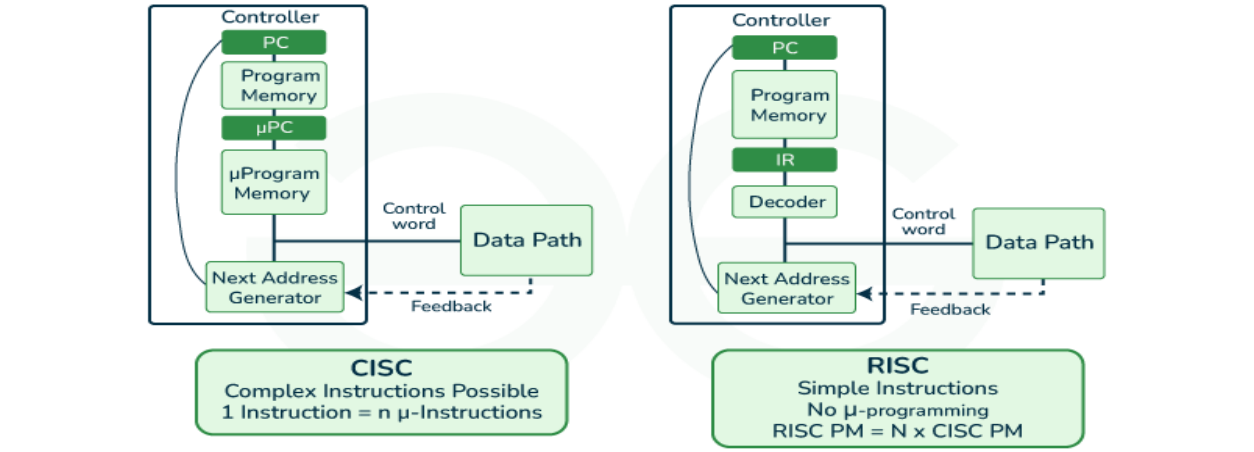
Both approaches try to increase the CPU performance

RISC: Reduce the cycles per instruction at the cost of the number of instructions per program.

$$CPU\ Time = \frac{Seconds}{Program} = \frac{Instructions}{Program} X \frac{Cycles}{Instructions} X \frac{Seconds}{Cycle}$$

CPU Time

CISC: The CISC approach attempts to minimize the number of instructions per program but at the cost of an increase in the number of cycles per instruction.



Advantages of CISC

- Reduced code size: CISC processors use complex instructions that can perform multiple operations, reducing the amount of code needed to perform a task.
- More memory efficient: Because CISC instructions are more complex, they require fewer instructions to perform complex tasks, which can result in more memory-efficient code.
- Widely used: CISC processors have been in use for a longer time than RISC processors, so they have a larger user base and more available software.

Disadvantages of CISC

- Slower execution: CISC processors take longer to execute instructions because they have more complex instructions and need more time to decode them.
- More complex design: CISC processors have more complex instruction sets, which makes them more difficult to design and manufacture.
- Higher power consumption: CISC processors consume more power than RISC processors because of their more complex instruction sets.

RISC-V INSTRUCTIONS

RISC-V is an open-source instruction set architecture used to develop custom processors for a variety of applications, from embedded designs to supercomputers.

RISC-V has gained popularity because the architecture provides simplified instructions to the processor to accomplish various tasks. It also enables designers to create thousands of potential custom processors, facilitating faster time to market.

RISC-V Machine Model

RISC-V is not actually one instruction set, but it is a collection of instruction sets that incrementally build on top of each other.

- RISC-V has 32 32-bit registers
- Registers are faster than memory
- RISC-V called “32-bit architecture” because it operates on 32-bit data

C Code

```
a = b + c;

a = b - c;
```

RISC-V assembly code

add a, b, c

sub a, b, c

- Add/sub: mnemonic indicates operation to perform
- b, c: source operands (on which the operation is performed)
- a: destination operand (to which the result is written)

More complex code is handled by multiple RISC-V instructions.

C Code

a = b + c - d;

RISC-V assembly code

add t, b, c # t = b + c

sub a, t, d # a = t - d

RISC-V RV32I has six types of instructions.

- type (Register/register) instructions use only registers as source and destination. This instruction type is mostly used for arithmetic and logic operations involving the ALU.
- I-type (Immediate) instructions has one of the two source operands specified within the 32-bit instruction word as a 12-bit constant (or immediate). This constant is regards as 12-bit signed 2's complement number, which is always sign extended to form a 32-bit operand.
- S-type (Store) instructions are exclusively used for storing contents of a register to data memory.
- B-type (Branch) instructions are used to control program flow. It compares two operands stored in registers and branch to a destination address relative to the current Program Counter value.
- J-type (Jump) instructions are used for subroutine calls.
- U-type (Upper immediate) instructions are used to specify the upper 20 bits immediate value of a register.

x86 Instruction Set

Intel x86 architecture has evolved over the years. From a 29, 000 transistors microprocessor 8086 that was the first introduced to a quad-core Intel core 2 which contains 820 million transistors, the organization and technology have changed dramatically. The updated instruction set is grouped according to architecture (i186, i286, i386, i486, i586/i686) and is referred to as (32-bit) x86 and (64-bit) x86-64. Some of the highlights of the evolution of x86 architecture are:

8080 – It was the world's first general-purpose microprocessor. It was an 8-bit machine, with an 8-bit data path to memory. It was used in the first personal computer.

8086 – It was a 16-bit machine and was far more powerful than the previous one. It had a wider data path of 16-bits and larger registers along with an instruction cache or queue that prefetches a few instructions before they are executed. It is the first appearance of 8086 architecture. It has a real mode and an addressable memory of 1 MB.

80286 – It has an addressable memory of 16 MB instead of just 1 MB and contains two modes-real mode and first-generation 16-bit protected mode. It has a data transfer width of 16-bits and a programming model of 16-bits (16-bits general purpose registers and 16-bit addressing).

80386 – It was Intel's first 32-bit machine. Due to its 32-bit architecture, it was able to compete against the complexity and power of microcomputers and mainframes introduced just a few years earlier. It was the first processor to support multitasking and contained the 32-bit protected mode. It also implemented the concept of paging (permitted 32-bit virtual memory address to be translated into 32-bit physical memory address). It has an addressable physical memory of 4 GB and a data transfer width of 32 bits.

80486 – It introduced the concept of cache technology and instruction pipelining. It contained a write protect feature and offered a built-in math co-processor that offloaded complex math operations from the main CPU.

Pentium – The use of superscalar techniques was introduced as multiple instructions started executing in parallel. The page size extension (PSE) feature was added as a minor enhancement in paging.

Pentium Pro – It used register renaming, branch prediction, data flow analysis, speculative execution, and more pipeline stages. Advanced optimization techniques in microcode were also added along with level 2 cache. It implemented the second-generation address translation in which a 32-bit virtual address is translated into a 36-bit physical memory address.

Pentium II – It was able to process video, audio, and graphics data efficiently by incorporating Intel MMX technology (multimedia data set).

Pentium III – It contains SMD (streaming extensions) instructions (SSE) and supports 3D graphics software. It has a maximum CPU clock rate of 1.4 GHz and contained 70 new instructions.

Pentium 4 – It implements third-generation address translation that translates a 48-bit virtual memory address to a 48-bit physical memory address. It contains other floating point enhancements for multimedia.

Core – It is the first Intel microprocessor with dual-core which is the implementation of 2 processors on a single chip. There is an addition of Visualizing Technology.

Core 2 – It extends the architecture to 64 bits and core 2 Quad provides four processors on a single chip. The register set, as well as addressing modes, are 64 bits.